ATLANTECH

BACKEND ARCHITECTURE

One Laravel modular monolith on each port server, with Python AI workers beside it

Port server · Laravel API

Built in seven phases (P1–P7)

CLIENTS

Vue web dashboard

Office users: staff, officers, planners, Port Admin

· own plan, consumes OpenAPI

HTTPS / SSE

Flutter mobile app

Pilots, tug masters, terminal supervisors

Offline-first: local Outbox, idempotent sync events

· MDM phone, per-app VPN

HTTPS / WS

EDGE

NGINX

TLS 1.3 termination

app.atlantech.io via split DNS

· one build for all ports

daily licence check-in · outbound

Central SaaS (separate Laravel app, P7)

SaaS admin dashboard + SaaS API + SaaS DB

Port registry, licences, billing records

Check-in API signs Ed25519 tokens lazily

Never sees port operational data

· no runtime call into any port

Weather and marine

Primary + secondary providers

· called by Part 1

FCM / APNs

Content-free wake-up push

· post-MVP (FW-14)

Delivery phases

- P1 Repo, schema, models, login
- P2 Write chain, audit, users, master data
- P3 Requests, BAPLIE, AiBridge + mock
- P4 Window, recommendation, plan confirm
- P5 Dispatch, notifications, mobile sync
- P6 Yard, port licence, real AI, E2E
- P7 Central SaaS

LARAVEL PORT API · project-backend

HTTP layer

Routes /api/v1 · OpenAPI 3.0 | Sanctum: web session · device token | RFC 9457 errors (AR / EN) | Requests validated → 422

Write chain: every request passes these steps in order, each refusal is audited

Auth 401 → Password gate 403 → Role + terminal 403 → App version 426 → Licence mode 423 → Version token 409 → Business rules 409 / 422 → Plan limits 422

Domain modules (one folder each, one-way dependencies, terminal modules never import each other)

Access and licence

Access: users, roles, devices, terminal scope (P1–P2)

License: mode SETUP / FULL / READ_ONLY, check-in client, plan limits (P2, P6)

Master data and yard

MasterData: port, channel, terminals, berths, tugs, pilots, shifts, safety limits, tide sets (P2)

Yard: zones, blocks, slots, yard recommendation, TOS export (P6)

AiBridge P3

Job publisher: JSON envelope v1, RPUSH on an unprefixed connection

Run row first, same transaction; push after commit; sweeper re-enqueues

Completion consumer on queue:ai-results + poller safety net, stale-run handling

Contract tests vs JSON Schemas (CI)

Vessel call flow (core loop)

Berthing (request, SM-4, customs (P3)) → Baplie (upload, match, withdraw (P3)) → Environment (manual data, conditions (P4)) → Recommendation (review, override, regenerate (P4)) → Plan (confirm, cancel, re-plan (P4–P5))

request → window → recommendation → confirmed plan

Realtime and notifications P3 / P5

SSE stream to the dashboard (≤ 5 s)

Reverb WebSocket to the mobile app

notification_log row per send

Re-send T+10, T+20; escalate T+30

FCM wake-up push (post-MVP)

Field execution and learning

Dispatch: assign pilot, tug master, tug (P5) · Execution: sync events, status, clock-skew flag, deviations, delay re-plan (P5) · Learning: evaluation and drift data for the Port Admin (P6)

Scheduler and queue workers

Laravel jobs on Redis (supervised)

Run sweeper and stale-run sweep

Pre-execution reminders (2 h default)

Acknowledgement retries, escalation

Nightly licence check-in (random time)

· runs in READ_ONLY too (OD-18)

Shared kernel (used by every module)

Audit in the same transaction | SM-4 transitions (single writer) | ResourceChanged domain event

PHP enums = SQL status names | Terminal-scope query filters (403) | Version token on updated_at

PYTHON AI WORKERS · owned by the AI team

EDI Parser (B1)

BAPLIE D.17A / SMDG 2019, deterministic

All-or-nothing commit; PARSED, QUARANTINED (staged data) or FAILED

· queue:edi

AI Part 1 · predictive and environmental

Weather every 6 h, tide daily (pyTMD)

Surge = sea level − tide; UKC, squat

Feasible windows, blocked periods

Never allocates · queue:part1

· own internal scheduler

AI Part 2 · AI Engine

Hard filter, service time, delay risk

CP-SAT berth, tug, pilot (10 s)

FIFO only on technical failure

Yard placement · queue:part2

· only allocation authority

AI Part 3 · feedback loop

Evaluation and drift on trusted data (clock_skew_suspect = 0)

· local scheduler; retraining is future work

Workers read port state and write result tables directly (part1_run, solver_run, recommendation...). They never write request status, audit or plans.

Swap in for the mock in Phase 6

DATA LAYER

Eloquent — writes + audit, one txn

store file

RPUSH jobs

BLPOP

RPUSH queue:ai-results

MySQL 8 · port database

~90 tables, migrations owned by backend

Real CHECKs, generated columns, ENUMs

audit_log: INSERT + SELECT only

One DB account per AI worker

· primary; replica in production

BAPLIE file volume

Uploaded EDI files

SHA-256, live_hash_key

Read by EDI Parser

Model volume

Model artefacts (artifact_path)

Backed up with the server

Redis

Cache and Laravel jobs

queue:part1 · queue:part2

queue:edi (backend → AI)

queue:ai-results (AI → backend)

· plain JSON, no key prefix

read state · write results (per-worker DB account)

RULES THE ARCHITECTURE ENFORCES

- AI never writes request status: backend owns transitions, history and audit
- One vocabulary: SQL status names in PHP enums and in the AI contract
- Run row first: created in the same transaction, job pushed after commit
- Terminal isolation: AI outputs and manifests filtered by terminal (403)
- Every write passes the write chain and is audited in its own transaction
- Hard safety limits are never relaxed by any model or by FIFO